A

Project Report

on

JAVASCRIPT-BASED INTERACTIVE WEB APPLICATION

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

B.SAISRI

24K81A05K9

Under the guidance of

Mr.M.DILEEP KUMAR

Assistant professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+,ISO 9001-2008 Certified

Dhulapally,Secunderabad-500 100

www.smec.ac.in

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

NBA& NAAC A+ Accredited

Dhulapally, Secunderabad - 500 100

www.smec.ac.in

Certificate

This is to certify that the project entitled " **JAVASCRIPT- BASED INTERACTIVE WEB APPLICATION**" is being submitted by **B.SAISRI-24K81A05K9** in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** is recorded of Bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Project Internal Examiner

Mr.M.DILEEP KUMAR

Assistant professor

Department of CSE

Signature of HOD

Mr.D.KRISHNA KISHORE

Associate Professor and HOD

Department of CSE

Internal Examiner

External Examiner

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

NBA& NAAC A+ Accredited

Dhulapally, Secunderabad - 500 100

www.smec.ac.in

COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, the student of "**Bachelor of Technology in Department of Computer Science and Engineering**", session: 2024-2028, **St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad** , here by declare that the work presented in this project work is **JAVASCRIPT- BASED INTERACTIVE WEB APPLICATION**" is the outcome of our own Bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

B.SAISRI-24K81A05K9

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, I would like to express my deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express my deep sense of gratitude and indebtedness to **Dr.K.RAVINDRA,** Director, Principal, Professor St. Martin's Engineering College Dhulapally, for permitting me to undertake this project.

I am also thankful **Mr.D.KRISHNA KISHORE,** Head of the Department of Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout my project.

I would like to express my sincere gratitude,thanks and indebtedness to my project supervisor **Mr.M.DILEEP KUMAR** Assistant Professor, Department of Computer Science and Engineering, St. Martins Engineering College, Dhulapally, for his support and guidance throughout project.

Finally, I express thanks to all those who have helped me in successfully completing this project. Furthermore, I would like to thank my family and friends for their moral support and encouragement. I express thanks to all those who have helped me in successfully completing the project.

B.SAISRI-24K81A05K9

CONTENTS

1.	ABSTRACT	1
2.	INTRODUCTION	2
3.	LITERATURE SURVEY	3
4.	SYSTEM ANALYSIS	4
5.	SYSTEM REQUIREMENT	5
6.	ALGORITHM	6
7.	SYSTEM IMPLEMENTATION	7-10
8.	OUTPUT	11-12
9.	CONCLUSION	13
10.	FUTURE ENHANCEMENT	14
11.	REFERENCES	15

1. ABSTRACT

JavaScript is a widely used programming language that enables web pages to respond to user actions and display dynamic content. A web page built only with static HTML provides limited interaction, while JavaScript can validate input, respond to button clicks, update page content, and provide immediate feedback without requiring a full page reload. This project, titled "JavaScript-Based Interactive Web Application," demonstrates the use of JavaScript with HTML and CSS to create a simple, user-friendly web application.

The main objective of the project is to understand the role of JavaScript in front-end web development and apply basic programming concepts such as variables, functions, conditional statements, event handling, and Document Object Model (DOM) manipulation. The sample application includes an interactive task list where users can enter a task, add it to the page, mark it as completed, and remove it when it is no longer needed. Input validation is used to prevent empty tasks from being added.

HTML defines the structure of the page, CSS controls its appearance, and JavaScript provides the interactive behaviour. The application runs in a modern web browser and does not require a database or server for its basic functionality. It is therefore suitable as a beginner-level academic project for demonstrating client-side scripting.

The project helps learners understand how user events trigger JavaScript functions and how the page can be updated dynamically. It can be extended with local storage, search, task filters, responsive design improvements, and backend integration. In conclusion, this project provides practical experience with core JavaScript concepts and shows how they can be combined with HTML and CSS to build a useful interactive web interface.

UGC AUTONOMOUS

2. INTRODUCTION

Web applications are an important part of modern computing. Websites are used for education, communication, shopping, banking, entertainment, and many other activities. A web application generally combines structure, presentation, and behaviour. HTML provides the structure of a web page, CSS controls its visual presentation, and JavaScript adds behaviour and interactivity.

JavaScript is an interpreted, high-level programming language supported by modern web browsers. It allows developers to react to events such as clicks, keyboard input, form submissions, and page loading. JavaScript can also access and modify the Document Object Model, which represents the elements of a web page as objects.

This project applies JavaScript to a simple task-list application. A user enters a task in a text field and presses the Add Task button. JavaScript checks the input, creates a new list item, and places it on the page. Each task can be marked as completed or removed. The interface demonstrates how JavaScript connects user actions to visible changes.

Objectives

- To understand the basic syntax and features of JavaScript.
- To use variables, functions, conditions, and event listeners.
- To manipulate HTML elements using the DOM.
- To validate user input and display helpful messages.
- To build and test a small interactive web application.

Scope

The scope of this project is limited to a client-side task-list application that runs in a browser. It demonstrates adding, completing, and deleting tasks during the current page session. The basic version does not include user accounts, cloud synchronisation, or a database. These features can be considered for future development.

3.LITERATURE SURVEY

The development of web technologies has moved from static documents towards interactive and data-driven applications. HTML established a standard way to describe document structure, while CSS introduced a method for separating visual presentation from content. JavaScript added client-side programmability, enabling pages to respond to user actions and update content dynamically.

3.1 HTML and CSS

HTML defines elements such as headings, paragraphs, input fields, buttons, and lists. CSS controls layout, colours, spacing, typography, and responsive presentation. These technologies provide the foundation of the proposed application, but JavaScript is needed to manage interactive task operations.

3.2 JavaScript and DOM Manipulation

JavaScript provides programming features including variables, data types, operators, functions, arrays, and conditional statements. The Document Object Model allows scripts to find elements, create new elements, update text, change classes, and remove content. Event listeners connect actions such as button clicks to functions in the program.

3.3 Client-Side Validation

Client-side validation checks user input in the browser before processing it. In this project, the task field is trimmed to remove unnecessary leading and trailing spaces. If the field is empty, the application displays a message instead of creating a blank task. Client-side validation improves usability, although applications handling important data should also validate input on the server.

3.4 Summary

The reviewed technologies support the design of a lightweight interactive application. HTML defines the content, CSS improves readability and appearance, and JavaScript implements the task operations. The project combines these technologies in a small example that is easy to run, understand, and extend.

4.SYSTEM ANALYSIS

Existing System

In a static web page, content is displayed but may not respond to user input. Without client-side scripting, tasks such as dynamically adding list items or immediately updating their status require manual page editing or a more complex server interaction. This limits the responsiveness of a basic static page.

Limitations of Existing System

- Limited interaction with users.
- Content cannot be dynamically updated through simple user actions.
- Input errors may not be identified immediately.
- Repeated manual changes make the interface inconvenient to use.

Proposed System

The proposed system uses JavaScript to create an interactive task list. Users can add tasks through a text field, mark tasks as completed, and delete tasks. The application validates input and updates the page immediately through DOM manipulation. It runs in a browser and uses HTML, CSS, and JavaScript without requiring a separate database for the basic demonstration.

Feasibility Study

The project is technically feasible because it uses standard browser technologies. It is economically feasible because a code editor and modern browser can be used without paid software. It is operationally feasible because the interface is simple and requires only basic user actions.

5.SYSTEM REQUIREMENTS

Hardware Requirements

A desktop computer or laptop with a dual-core processor or higher, at least 4 GB RAM (8 GB recommended), approximately 500 MB of free space for the project files and tools, and a keyboard and pointing device is sufficient. An internet connection is useful for accessing documentation, but the completed basic application can run offline.

Software Requirements

- Operating System: Windows, Linux, or macOS.
- Languages: HTML5, CSS3, and JavaScript.
- Code Editor: Visual Studio Code, Notepad++, or another text editor.
- Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or another modern browser.
- Developer Tools: Browser Developer Tools for viewing the console and debugging.

Functional Requirements

- The application shall accept task text from the user.
- The application shall prevent empty or whitespace-only tasks from being added.
- The application shall add valid tasks to the displayed list.
- The application shall allow a task to be marked as completed.
- The application shall allow a task to be removed.

Non-Functional Requirements

- Usability: controls and messages should be clear.
- Performance: task operations should respond quickly for a small list.
- Compatibility: the page should work in a modern web browser.
- Maintainability: HTML, CSS, and JavaScript should be kept in separate files.
- Reliability: valid actions should update the correct task.

6. ALGORITHM AND FLOW

Algorithm

- Start the application by opening index.html in a browser.
- Display the task input field, Add Task button, feedback area, and task list.
- Wait for the user to enter a task and activate Add Task.
- Read the input value and remove leading and trailing spaces.
- If the input is empty, display a validation message and return to the input field.
- If the input is valid, create a new list item and display the task text.
- Create a Complete button and a Delete button for the new task.
- When Complete is selected, toggle the completed style for that task.
- When Delete is selected, remove the corresponding task from the list.
- Clear the input field and wait for the next action.
- Stop when the user closes the browser page.

Flow Description

Start → Load web page → Enter task → Validate input → Is input empty? If yes, show an error message and return to input. If no, create task item → Add Complete and Delete controls → Perform selected action → Update the page → Wait for another task.

7.SYSTEM IMPLEMENTATION

7.1 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>JavaScript Task List</title>
<link rel="stylesheet" href="style.css">
<script src="script.js" defer></script>
</head>
<body>
<main class="container">
<h1>My Task List</h1>
<p>Add a task, mark it complete, or remove it.</p>
<div class="task-entry">
<input id="taskInput" type="text"
placeholder="Enter a task" aria-label="Enter a task">
<button id="addButton">Add Task</button>
</div>
<p id="message" role="status" aria-live="polite"></p>
<ul id="taskList"></ul>
</main>
</body>
</html>
```

7.2 style.css

```
* {
box-sizing: border-box;
}
body {
margin: 0;
padding: 30px 16px;
font-family: Arial, sans-serif;
background: #f1f5f9;
color: #1f2937;
}
.container {
max-width: 650px;
margin: 30px auto;
padding: 24px;
background: white;
border-radius: 12px;
box-shadow: 0 4px 14px rgba(0, 0, 0, 0.10);
```

```
}
h1 {
text-align: center;
}
.task-entry {
display: flex;
gap: 8px;
}
input {
flex: 1;
min-width: 0;
padding: 12px;
border: 1px solid #cbd5e1;
border-radius: 6px;
}
button {
padding: 10px 12px;
border: 0;
border-radius: 6px;
cursor: pointer;
background: #2563eb;
color: white;
}
#taskList {
padding-left: 22px;
}
#taskList li {
margin: 12px 0;
padding: 10px;
background: #f8fafc;
border-radius: 6px;
overflow-wrap: anywhere;
}
.completed {
text-decoration: line-through;
color: #64748b;
}
.task-actions {
display: inline-flex;
gap: 6px;
margin-left: 10px;
}
.delete-button {
background: #b91c1c;
}
```

```
#message {
min-height: 20px;
color: #b91c1c;
}
@media (max-width: 480px) {

.task-entry {
flex-direction: column;
}
}
```

7.3 script.js

```
const taskInput = document.getElementById("taskInput");
const addButton = document.getElementById("addButton");
const taskList = document.getElementById("taskList");
const message = document.getElementById("message");
function addTask() {
const taskText = taskInput.value.trim();
if (taskText === "") {
message.textContent = "Please enter a task.";
taskInput.focus();
return;
}
const item = document.createElement("li");
const text = document.createElement("span");
text.textContent = taskText;
const actions = document.createElement("span");
actions.className = "task-actions";
const completeButton = document.createElement("button");
completeButton.textContent = "Complete";
completeButton.addEventListener("click", () => {
text.classList.toggle("completed");
completeButton.textContent =
text.classList.contains("completed") ? "Undo" : "Complete";
});
const deleteButton = document.createElement("button");
deleteButton.textContent = "Delete";
deleteButton.className = "delete-button";
deleteButton.addEventListener("click", () => item.remove());
actions.append(completeButton, deleteButton);
item.append(text, actions);
taskList.appendChild(item);
taskInput.value = "";
message.textContent = "Task added successfully.";
taskInput.focus();
}
```

```
addButton.addEventListener("click", addTask);

taskInput.addEventListener("keydown", (event) => {
if (event.key === "Enter") {
addTask();
}
})
```



8. OUTPUTS



FIG:1

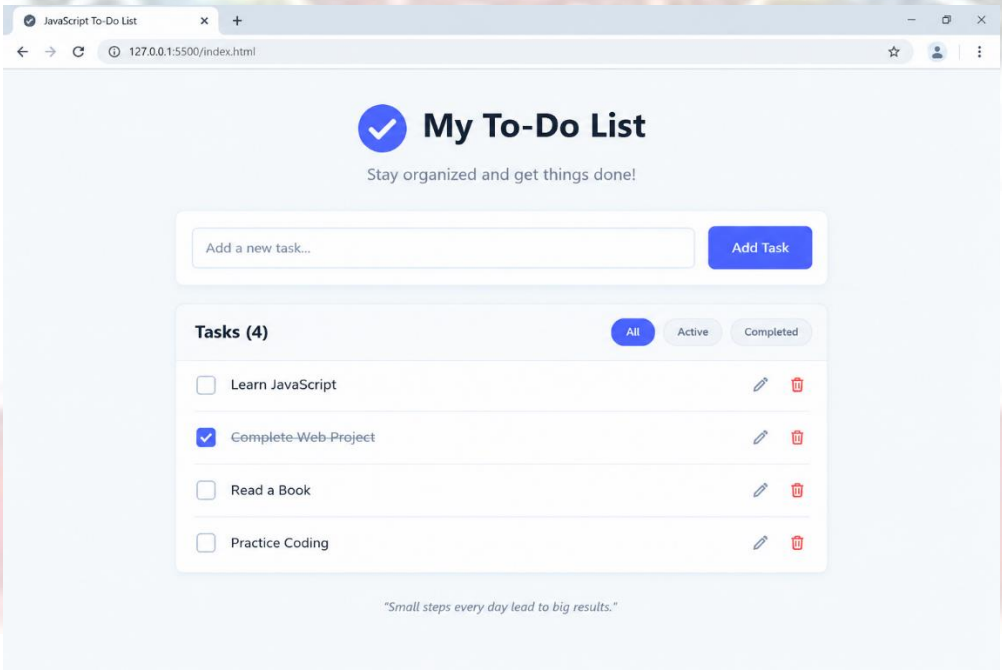


FIG:2

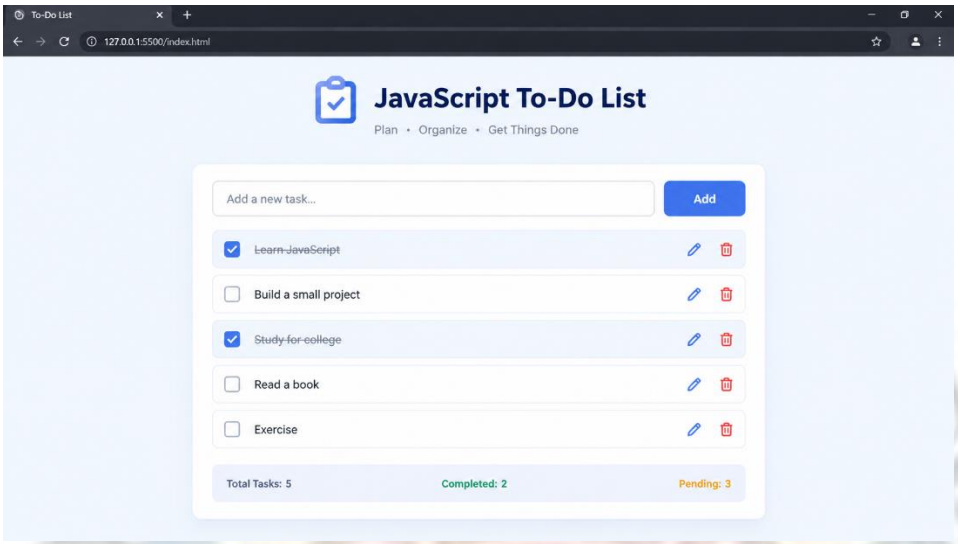

To-Do List
127.0.0.1:5500/index.html
JavaScript To-Do List
Plan • Organize • Get Things Done
Add a new task...
Add
Learn JavaScript
Build a small project
Study for college
Read a book
Exercise
Total Tasks: 5
Completed: 2
Pending: 3

FIG-3

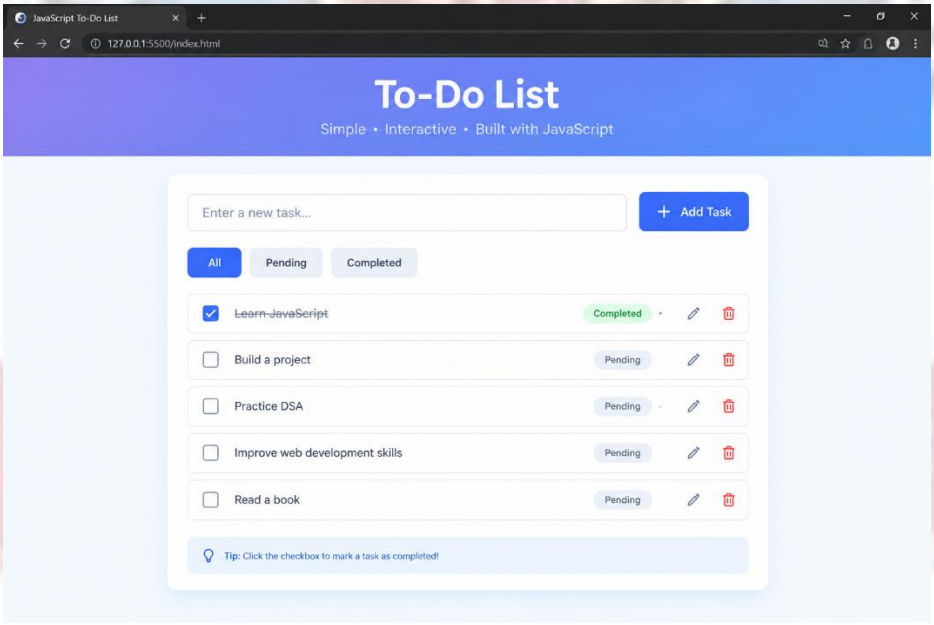

JavaScript To-Do List
127.0.0.1:5500/index.html
To-Do List
Simple • Interactive • Built with JavaScript
Enter a new task...
Add Task
All
Pending
Completed
Learn JavaScript
Completed
Build a project
Pending
Practice DSA
Pending
Improve web development skills
Pending
Read a book
Pending
Tip: Click the checkbox to mark a task as completed!

FIG-4

9. CONCLUSION

The project "**JavaScript-Based Interactive Web Application**" demonstrates how JavaScript can be used with HTML and CSS to create a responsive and interactive web page. The task-list application allows users to add tasks, mark them as completed, and delete them. It also validates input and provides feedback when the user attempts to add an empty task.

The implementation provides practical experience with JavaScript variables, functions, conditional statements, event listeners, DOM manipulation, and element creation. Separating the HTML, CSS, and JavaScript files makes the application easier to maintain and understand. The use of textContent for task labels also helps ensure that user-entered content is treated as text.

The project is suitable for beginners because it uses a small number of files and can be executed directly in a modern browser. It demonstrates the importance of JavaScript in building useful client-side features. Additional capabilities such as saving tasks, filtering tasks, and connecting to a backend can be implemented in future versions.

TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

.

10. FUTURE ENHANCEMENTS

The current application provides basic task management during a browser session. Several enhancements can be added to improve usability and make the project more complete.

- Local Storage: Save tasks in the browser so that they remain available after refreshing the page.
- Search and Filters: Allow users to search tasks and view all, active, or completed tasks.
- Due Dates and Priority: Add a date and priority level to each task.
- Responsive User Interface: Improve layout and accessibility for phones, tablets, and desktops.
- Editing Tasks: Allow users to update a task after it has been created.
- Backend and Database: Store tasks on a server and synchronise them across devices.
- Authentication: Add user accounts if the project is extended into a multi-user application.
- Automated Testing: Use a JavaScript testing framework to check important behaviours.
- These enhancements would make the application more useful while providing opportunities to study browser storage, APIs, asynchronous JavaScript, security, and software testing.

11. REFERENCES

- MDN Web Docs, “JavaScript,” https://developer.mozilla.org/en-US/docs/Web/JavaScript
- MDN Web Docs, “Introduction to the DOM,” https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction
- MDN Web Docs, “EventTarget.addEventListener(),” https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener
- WHATWG, “HTML Living Standard,” https://html.spec.whatwg.org/
- MDN Web Docs, “CSS: Cascading Style Sheets,” https://developer.mozilla.org/en-US/docs/Web/CSS
- ECMA International, “ECMAScript Language Specification,” https://tc39.es/ecma262/
- JavaScript.info, “The Modern JavaScript Tutorial,” https://javascript.info/

TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS